
DTU

02159 – Operating Systems

Week 4: Processes, Threads, and Scheduling

Xenofon (Fontas) Fafoutis
Professor
xefa@dtu.dk
www.compute.dtu.dk/~xefa

Roberto Morabito
Associate Professor
romor@dtu.dk
https://robertmora.github.io/

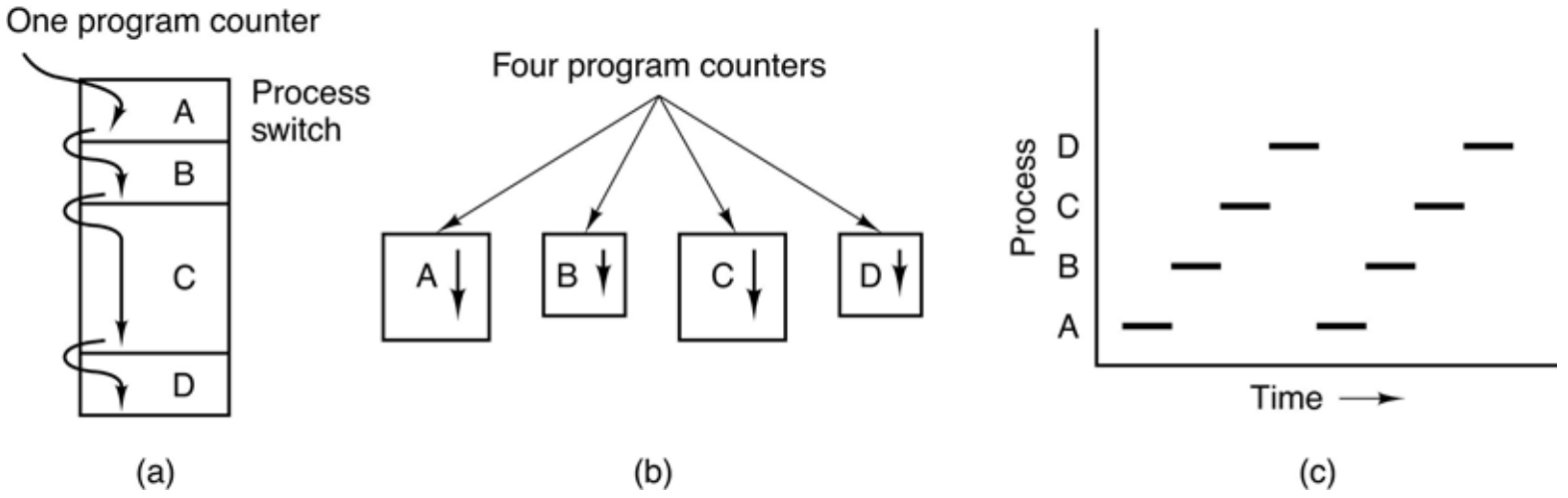
Slides originally created by Fontas Fafoutis and complemented by Roberto Morabito

Today's Topics

- Processes
- Process Scheduling
- Threads (and Thread Scheduling)

The Process Model

- **A Sequential Process**: An abstraction for executing a programme
  - The CPU is shared among **multiple processes** that run in parallel
    - **Pseudo-parallel** when the system has one CPU
    - The CPU is managed by the **scheduler**
  - The scheduler switches them (**context switch**)
    - Each process has its **logical PC** and its own **virtual** CPU



What is running on my laptop right now?

```
ps -A | wc -l
```

```
robertomorabito@Robertos-MacBook-Pro ~ % ps -A | wc -l
     586
```

```
ps -Ao pid,ppid,state,%cpu,comm | head -5
```

```
robertomorabito@Robertos-MacBook-Pro ~ % ps -Ao pid,ppid,state,%cpu,comm | head -5
  PID  PPID STAT  %CPU COMM
    1     0 Ss     0.1 /sbin/launchd
  330     1 Ss     0.1 /usr/libexec/logd
  332     1 Ss     0.0 /usr/libexec/UserEventAgent
  334     1 Ss     0.3 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/FSEvents.framework/Versions/A/Support/fseventsd
```

```
ps -Ao pid,ppid,state,%cpu,comm | sort -k4 -nr | head
```

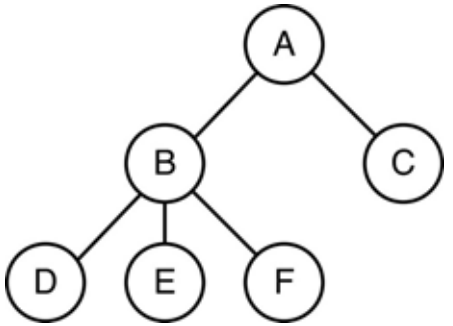
```
robertomorabito@Robertos-MacBook-Pro ~ % ps -Ao pid,ppid,state,%cpu,comm | sort -k4 -nr | head
63313     1 Ss     9.1 /System/Library/PrivateFrameworks/SkyLight.framework/Resources/WindowServer
27781     1 S      7.2 /Applications/NordVPN.app/Contents/MacOS/NordVPN
26795     1 S      5.7 /Applications/Adobe Acrobat DC/Adobe Acrobat.app/Contents/MacOS/AdobeAcrobat
86250 26739 R      5.2 /Applications/Google Chrome.app/Contents/Frameworks/Google Chrome Framework.fr
```

Question

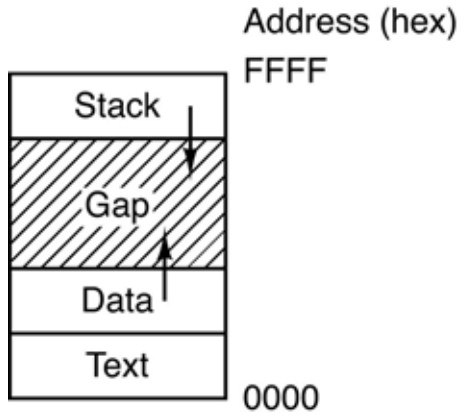
If I run my text editor **10 times** to edit 10 different files, how many processes do I get?

Process Creation

- In Unix, processes can only be created with the **fork** system call
  - **Fork** copies the calling process (**parent**) to a new process (**child**)
  - The child process is an **exact copy** of the parent
  - A process has **1 parent** and **0, 1 or more** children
  - All processes originate from the **init** process



- New programmes can be initiated in two steps:
  - A new process is created with the **fork** system call
  - The child process uses the **exec** system call to replace its code with another executable



- Parent and child have their own distinct **address space**
  - No **writable memory** is shared between them
  - For efficiency, the OS may use **copy-on-write**

Process Creation – fork() example

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Before fork: PID = %d\n", getpid());

    pid_t pid = fork();

    printf("After fork:  PID = %d, fork returned %d\n",
           getpid(), pid);

    return 0;
}
```

Process Creation – fork() example

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Before fork: PID = %d\n", getpid());

    pid_t pid = fork();

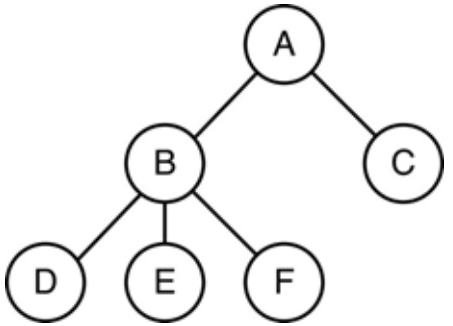
    printf("After fork:  PID = %d, fork returned %d\n",
           getpid(), pid);

    return 0;
}
```

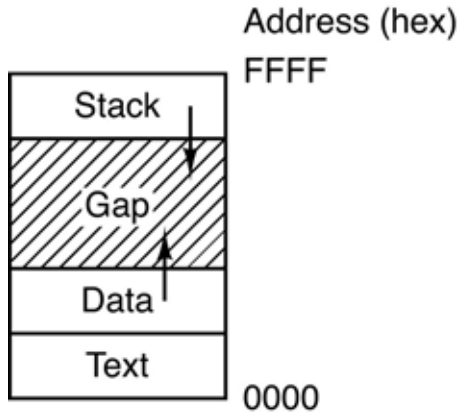
```
Before fork: PID = 23802
After fork:  PID = 23802, fork returned 23803
After fork:  PID = 23803, fork returned 0
```

Process Creation

- In Unix, processes can only be created with the **fork** system call
  - **Fork** copies the calling process (**parent**) to a new process (**child**)
  - The child process is an **exact copy** of the parent
  - A process has **1 parent** and **0, 1 or more** children
  - All processes originate from the **init** process



- New programmes can be initiated in two steps:
  - A new process is created with the **fork** system call
  - The child process uses the **exec** system call to replace its code with another executable

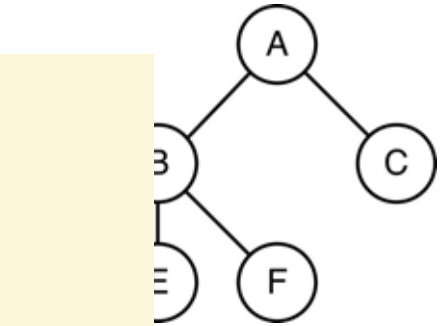


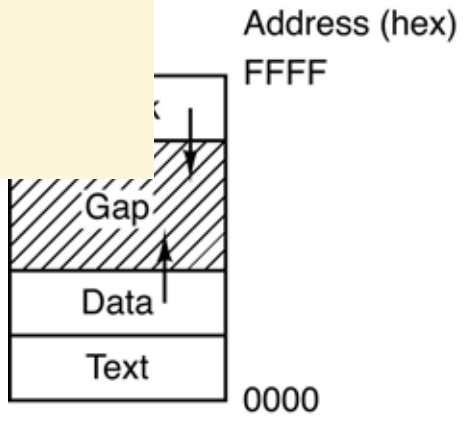
- Parent and child have their own distinct **address space**
  - No **writable memory** is shared between them
  - For efficiency, the OS may use **copy-on-write**

Process Creation

- In Unix, processes can only be created with the **fork** system call
  - **Fork** copies the calling process (**parent**) to a new process (**child**)
  - The child process is an **exact copy** of the parent
  - A process has **1 parent** and **0, 1 or more** children
  -
- New
  -
  -
- Pare
  - No **writable memory** is shared between them
  - For efficiency, the OS may use **copy-on-write**

```
shell
  |
 fork()
  |
  +---- child
          |
        exec("ls")
```





Process Creation – fork() example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    int *A = malloc(sizeof(int));
    *A = 50;

    pid_t pid = fork();

    *A = *A + 1;

    printf("PID=%d, A=%d\n", getpid(), *A);

    return 0;
}
```

```
PID=43913, A=51

PID=43914, A=51
```

Process Creation – fork() example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    int *A = malloc(sizeof(int));
    *A = 50;

    pid_t pid = fork();

    *A = *A + 1;

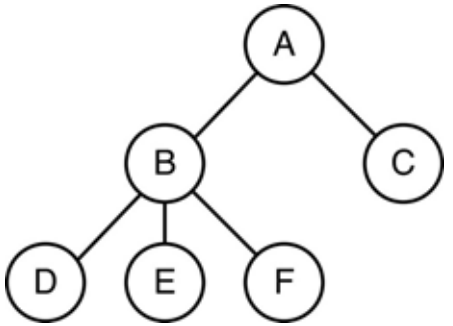
    printf("PID=%d, address=%p, A=%d\n", getpid(), (void *)A, *A);
    return 0;
}
```

```
PID=43913, address=0x105581a00, A=51

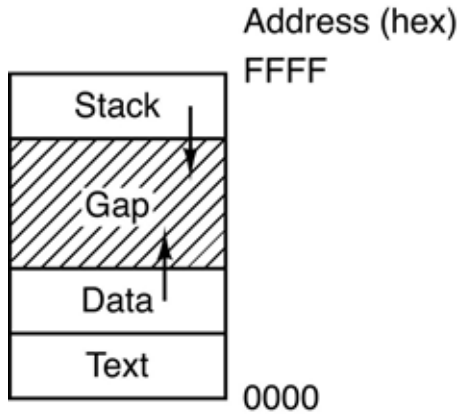
PID=43914, address=0x105581a00, A=51
```

Process Creation

- In Unix, processes can only be created with the **fork** system call
  - **Fork** copies the calling process (**parent**) to a new process (**child**)
  - The child process is an **exact copy** of the parent
  - A process has **1 parent** and **0, 1 or more** children
  - All processes originate from the **init** process



- New programmes can be initiated in two steps:
  - A new process is created with the **fork** system call
  - The child process uses the **exec** system call to replace its code with another executable



- Parent and child have their own distinct **address space**
  - No **writable memory** is shared between them
  - For efficiency, the OS may use **copy-on-write**

Process Termination

- A process terminates **voluntarily** by calling the **exit** system call
  - **Normal** exit or **Error** exit is indicated by a exit **status code**

- A process can also be terminated **involuntarily**
  - Killed by another process (**signals**)
  - **Fatal error** (illegal instruction, division by zero, etc)

Process Table

- The OS maintains a table of all processes (**process table**)

Process management	**Memory management**	**File management**
Registers	Pointer to text segment info	Root directory
Program counter	Pointer to data segment info	Working directory
Program status word	Pointer to stack segment info	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

Process Table

- The OS maintains a table of all processes (**process table**)

Process management	**Memory management**	**File management**
Registers	Pointer to text segment info	Root directory
Program counter	Pointer to data segment info	Working directory
Program status word	Pointer to stack segment info	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

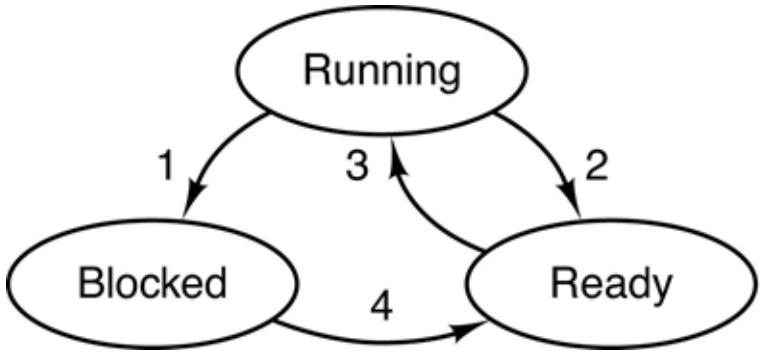
Q: What are the possible states?

Process States

- An active process can be in **three** possible states
- **Running State**: Actively using the CPU
- **Ready State**: Runnable, but suspended
- **Blocked State**: Unable to run until some external event happens

Q: When does a process change state?

Process State Transitions



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

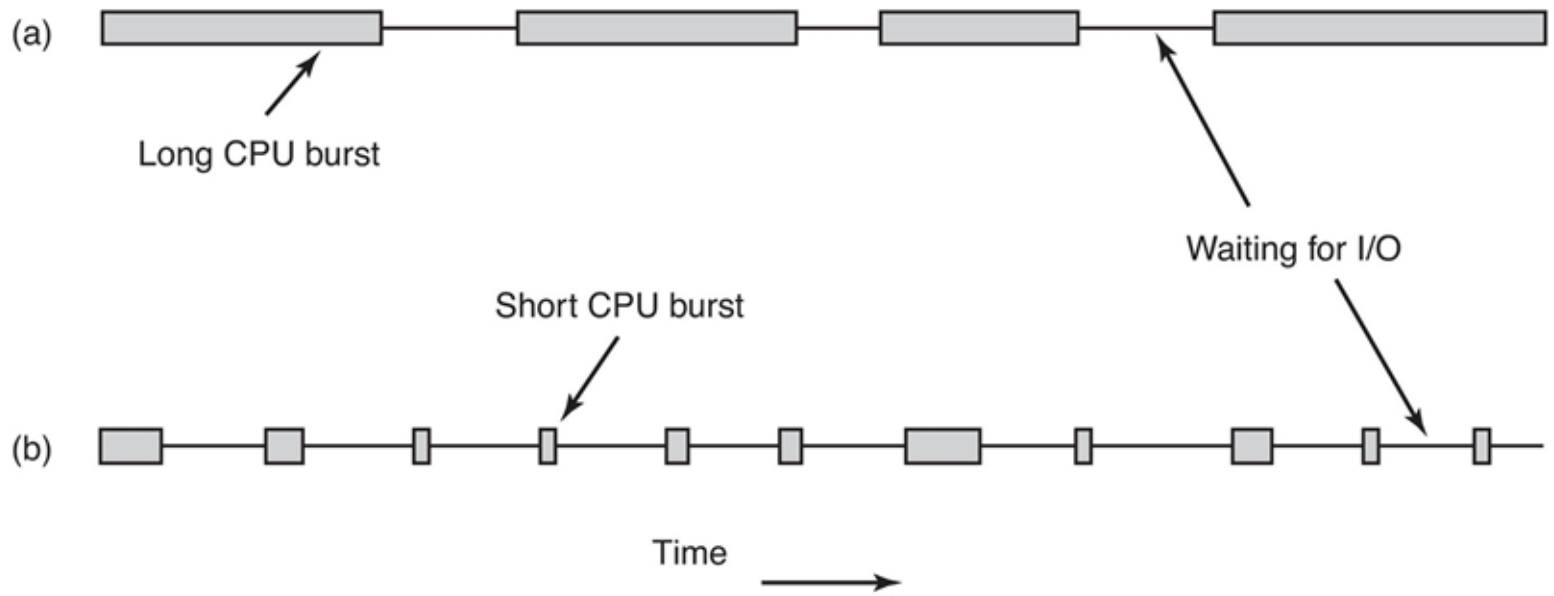
Can We See the States?

```
ps -Ao pid,state,%cpu,comm
```

- The textbook gives us a model
  - Running, Ready, Blocked. A real OS has a richer implementation.

Process Behaviour

- A process alternates bursts of **computing** and **I/O requests**
  - Note: Computing is defined as executing instructions
- A **CPU-bound** process spends most of its time computing
- A **I/O-bound** process spends most of its time waiting



Process Behaviour

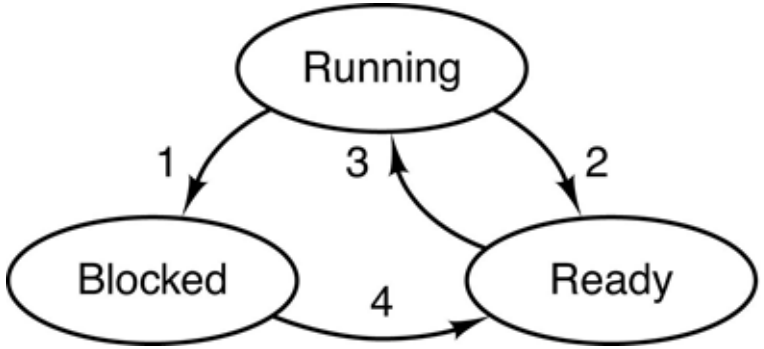
```
top
```

```
Processes: 526 total, 2 running, 524 sleeping, 5938 threads
Load Avg: 1.86, 2.77, 2.73  CPU usage: 5.35% user, 3.57% sys, 91.7% idle  SharedLibs: 556M resident, 116M data
PhysMem: 23G used (3998M wired, 8521M compressor), 123M unused. VM: 351T vsize, 6144M framework vsize, 3014065
Disks: 498936782/10T read, 271779150/4702G written.

PID    COMMAND      %CPU  TIME      #TH    #WQ  #PORTS MEM    PURG   CMPRS  PGRP  PPID  STATE    BOOSTS
63313  WindowServer 26.2  54:49:53 23     6    10478  1565M+ 36M-   496M-  63313 1     sleeping *0[1]
25049  top          13.6  00:04.30 1/1    0    34     8896K  0B     0B     25049 10075 running  *0[1]
0      kernel_task  9.6   53:16:18 779/14 0    0      49M-   0B     0B     0     0     running   0[0]
26795  AdobeAcrobat 7.1   58:51.00 70     4    1442-  932M-  0B     777M-  26795 1     sleeping  0[3932]
86250  Google Chrom 5.9   10:25.59 27     1    260    298M+  0B     170M-  26739 26739 sleeping *0[250497+]
27677  com.nordvpn. 4.8   09:07.77 32     5    222    23M    0B     12M    27677 1     sleeping *0[1]
```

Process Scheduling

- Only **one** process can be at a **running** state (per CPU)
- Multiple processes in the **ready** state compete for the CPU
- The decision is up to the **scheduler**



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

- The scheduler decides:
  - Which process (parent or child) to run after a **fork**
  - Which process to run after the running process calls **exit** (earlier `fork()` example)
  - Which process to run when a process gets **blocked** waiting for I/O
  - Which process to run when a process gets **unblocked** (i.e. ready)

Forcing a Running Process to Stop

- The **scheduler** must use the **CPU** to make decisions
- The CPU is **available** after a fork, exit, or when a process blocks
- How can the scheduler get the CPU from a **running process**?

Forcing a Running Process to Stop

- The **scheduler** must use the **CPU** to make decisions
- The CPU is **available** after a fork, exit, or when a process blocks
- How can the scheduler get the CPU from a **running process**?

```
while (1) {
}
```

Forcing a Running Process to Stop

- The **scheduler** must use the **CPU** to make decisions
- The CPU is **available** after a fork, exit, or when a process blocks
- How can the scheduler get the CPU from a **running process**?

Clock Interrupts

Preemptive vs Non-Preemptive Scheduling

- A **preemptive scheduler** allows a process to run up to **some time**
  - It sets a **clock interrupt** to occur at the end of the interval
  - The interrupt **stops** the running process and gives the CPU to the scheduler
  - Without hardware clocks, preemptive scheduling is not possible
  - In some cases interrupts can be disabled

- A **non-preemptive scheduler** lets the running process to run until it **blocks** or until it **exits** or until it **voluntarily** releases the CPU
  - Cooperative Scheduling

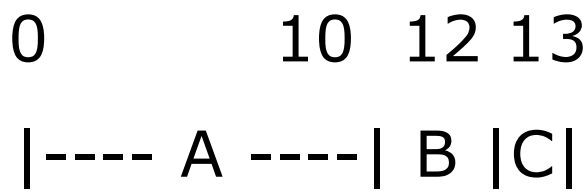
First-Come, First-Served (non-preemptive)

- Principle: CPU is given to the processes in the order they request it
- **Simple** implementation:
  - All **ready** processes are in a **FIFO**
  - The scheduler selects the process at the **beginning** of the queue
  - When a process becomes **ready**, it goes at the **end** of the queue

- Is it **fair**?

First-Come, First-Served (non-preemptive)

Process	Arrival	CPU burst
A	0	10
B	1	2
C	2	1



First-Come, First-Served (non-preemptive)

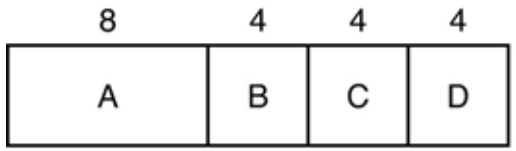
- Principle: CPU is given to the processes in the order they request it
- **Simple** implementation:
  - All **ready** processes are in a **FIFO**
  - The scheduler selects the process at the **beginning** of the queue
  - When a process becomes **ready**, it goes at the **end** of the queue

- Is it **fair**?

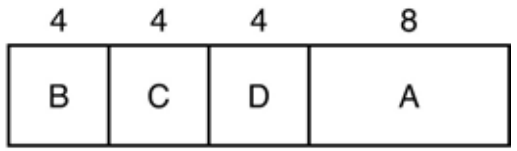
Short jobs wait for long jobs

Shortest Job First (non-preemptive)

- Principle: The scheduler selects the shortest process first
- Minimises the **mean turnaround time**



(a)

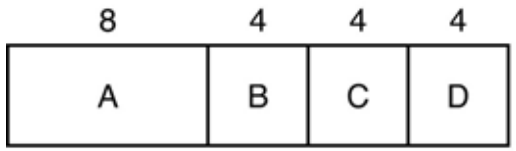


(b)

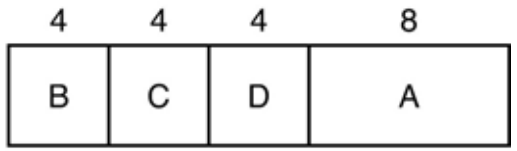
- What's the problem with that?

Shortest Job First (non-preemptive)

- Principle: The scheduler selects the shortest process first
- Minimises the **mean turnaround time**



(a)



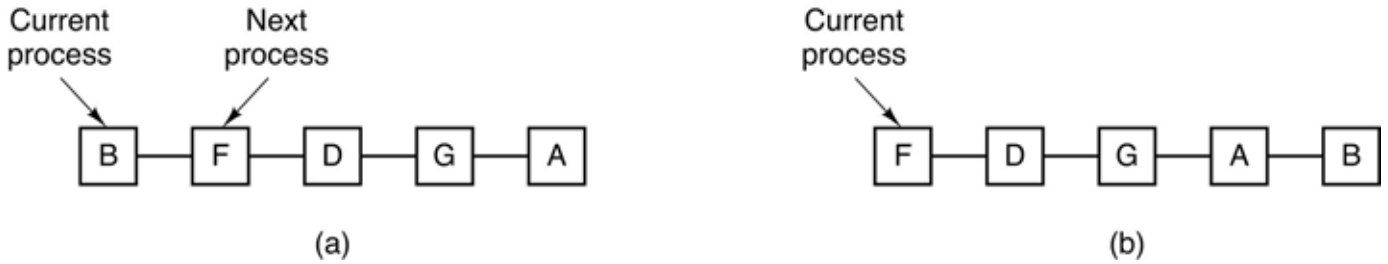
(b)

- What's the problem with that?

It's hard to know running times

Round-Robin Scheduling (preemptive)

- Principle: Each process is allowed to run for up to some fixed time
- **Simple** implementation:
  - All **ready** processes are in a **FIFO**, similar to FCFS
  - The scheduler defines a **time quantum**
  - If quantum is exceeded, the process is **stopped** and put at the **end** of the queue
  - If the process blocks, it also goes at the **end** of the queue

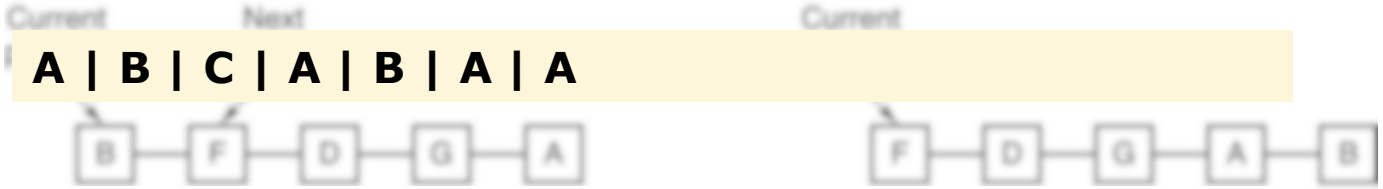


- How **long** shall the time quantum be?

Round-Robin Scheduling (preemptive)

- Principle: Each process is allowed to run for up to some fixed time
- Sim
  - A = 8 ms
  - B = 4 ms
  - C = 2 ms
  - quantum = 2 ms
  - t the
  - If the process blocks, it also goes at the **end** of the queue

A | B | C | A | B | A | A



What if quantum = 1000 seconds?

What if quantum = one CPU instruction?

- How **long** shall the time quantum be?

Round-Robin Scheduling (preemptive)

- How **long** shall the time quantum be?

Efficiency vs Delay Trade-off

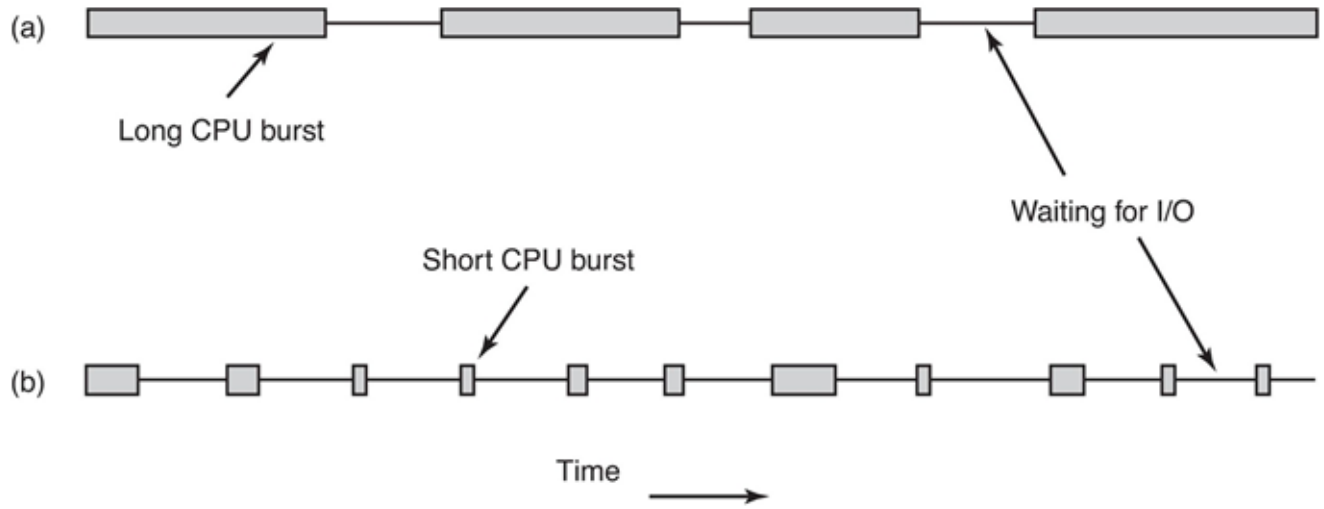
- Switching processes (**context switch**) take time
  - A very **short** quantum results to **high overhead**
- The quantum controls the response time
  - A very **long** quantum results to **high response time**

- Typically, 20-50 ms is a reasonable compromise

Are all processes equally important?

High Priority Processes

- Critical system-level process (e.g., scheduler)
- Interactive processes
  - Low response is important
  - User expectations matter
- I/O-bound processes



Priority Scheduling

- Principle: All processes are **not** equal
- Each process gets a **priority number**
- The scheduler selects the process with the **highest priority**

- Linux has 149 priority levels, 100 for the real-time, 49 for the user
  - Real-time processes do **not** get preempted (unless higher priority)
  - Processes with higher priority get **longer quantum**
  - A tool to voluntarily decrease the priority level: `nice`
    - `nice -n 10 yes > /dev/null &`
  - E.g. Some children do delay-tolerant tasks, some handle interaction with the user

- Dynamic priority levels
  - Set priority level to 1/f, where f is the % of quantum used

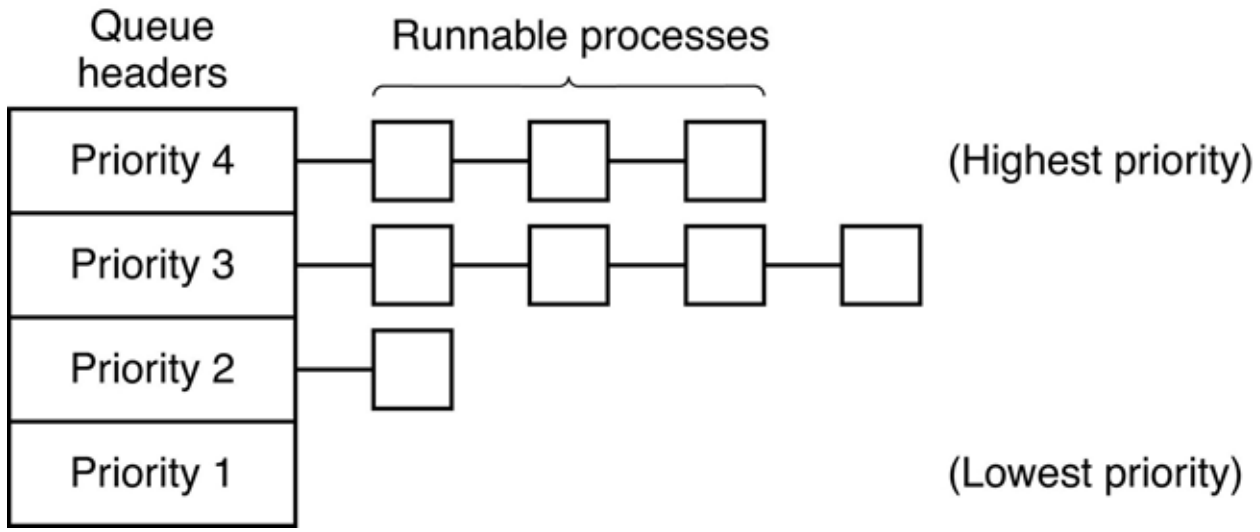
Priority Scheduling

```
top - 16:24:51 up  6:42,  1 user,  load average: 0.50, 0.87, 1.25
Tasks: 368 total,   2 running, 366 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.9 us,  0.9 sy,  0.0 ni, 97.0 id,  0.0 wa,  0.0 hi,  0.2 si,  0.0
MiB Mem :   7263.3 total,    406.4 free,   5184.3 used,   1672.6 buff/cache
MiB Swap:   2048.0 total,   1020.0 free,   1028.0 used.   1183.2 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+
   4363 sujal     20   0 1392.4g 277508  95492 S   7.9   3.7  12:20.29
   2082 sujal     20   0 5691876 222204  47476 S   6.9   3.0  18:06.91
   1893 sujal     20   0 2651860  43512  16860 S   4.6   0.6  16:43.02
   3938 sujal     20   0   34.0g 928972 695636 S   2.6  12.5  27:00.24
  15933 sujal     20   0 1392.4g 428260 137460 S   2.3   5.8   2:25.61
    272 root     -51   0       0      0      0 S   1.3   0.0   0:57.18
  10050 sujal     20   0   32.9g  90520  52968 S   1.0   1.2   8:08.11
  10122 sujal     20   0 1161.7g 203548  68644 S   1.0   2.7  22:49.50
   3985 sujal     20   0   33.1g 152804  93604 S   0.7   2.1  28:20.09
  14372 sujal     20   0 1392.5g 292652 142736 S   0.7   3.9   1:02.28
  14804 root      20   0       0      0      0 I   0.7   0.0   0:08.43
  16268 sujal     20   0   12120   3880   3124 R   0.7   0.1   0:00.15
    703 root      20   0       0      0      0 S   0.3   0.0   0:01.33
```

Priority Groups

- First, round-robin in priority group 4
- If group 4 is empty, round-robin in priority group 3
- Etc…



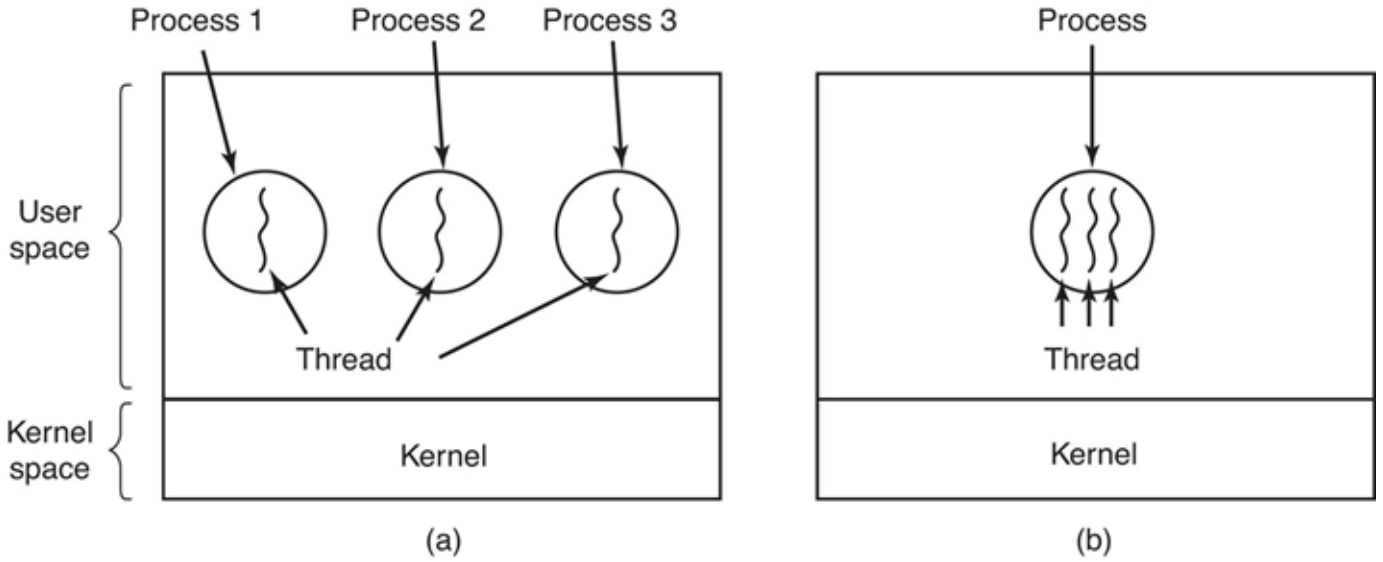
Other Scheduling Algorithms

- Lottery Scheduling
  - Each process gets a number of **lottery tickets**
  - Next process is selected **randomly**
  - Cooperative processes may exchange tickets

- Fair-share Scheduling
  - Processes have **owners**
  - In multi-user environments, we want fairness at user level

Threads

- Multithreading is a **second layer** of (pseudo-)parallelism
  - Traditionally, each process has a **single** thread of control
  - With multithreading, each process has **multiple** threads of control
- Different to processes, **threads share the same address space**
  - **Processes** are parallel entities in **different address spaces**
  - **Threads** are parallel entities in the **same address space**



Threads Share Resources

Per-process items	Per-thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

Threads Share Resources

Per-process items	Per-thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

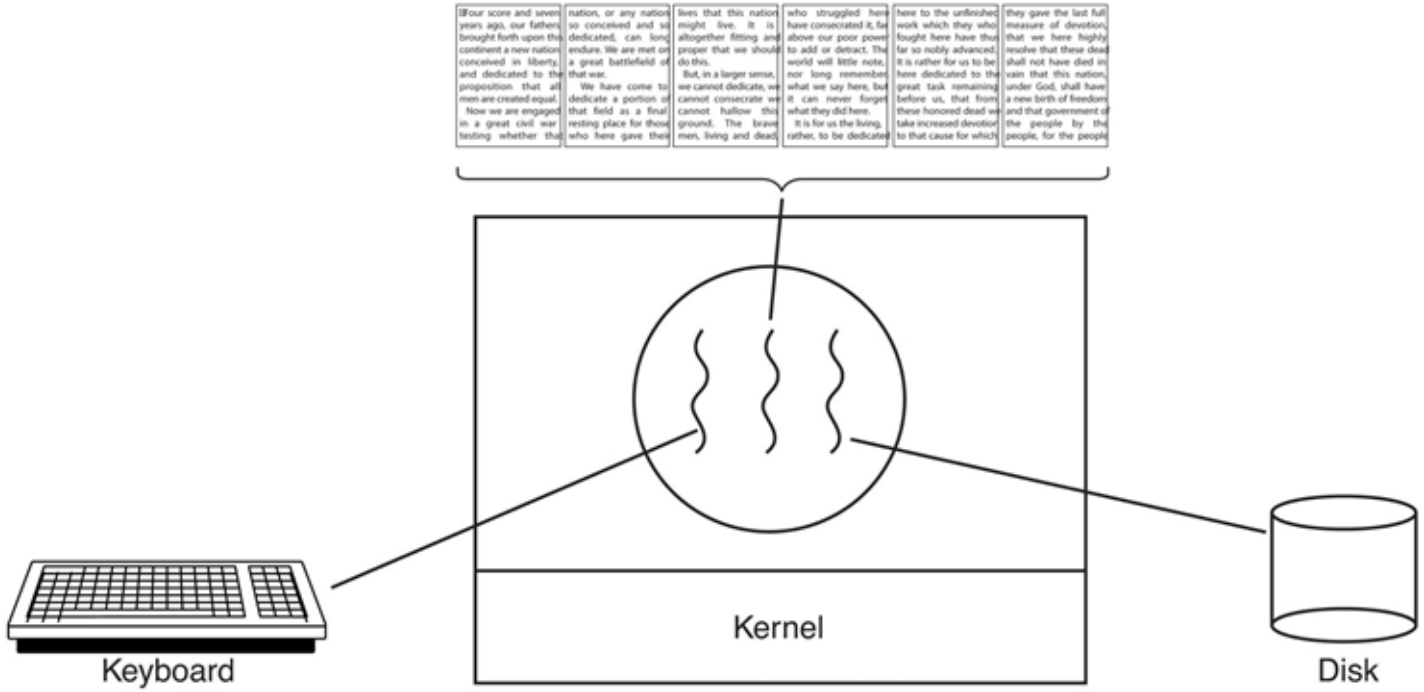
Shared process resources, private execution context.

POSIX Threads

Thread call	Description
Pthread_create	Create a new thread
Pthread_exit	Terminate the calling thread
Pthread_join	Wait for a specific thread to exit
Pthread_yield	Release the CPU to let another thread run
Pthread_attr_init	Create and initialize a thread's attribute structure
Pthread_attr_destroy	Remove a thread's attribute structure

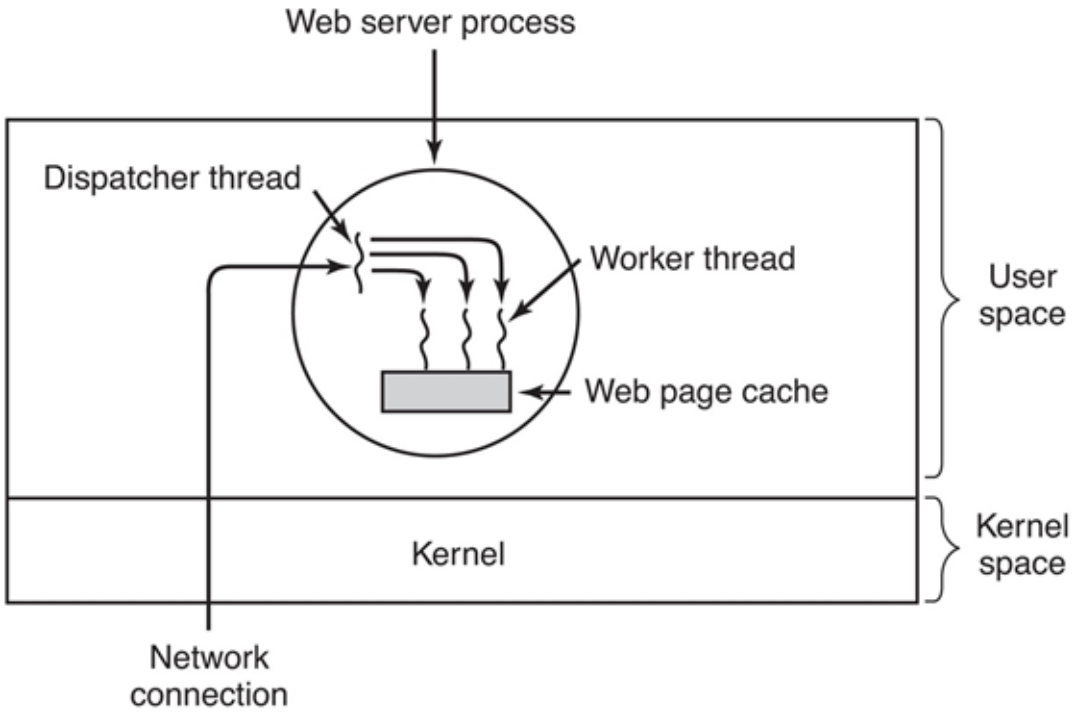
Thread Application Examples: Word Processor

- Threads are very convenient when parallel entities manipulate the **same memory**



Thread Application Example: Web Server

- Threads are more **lightweight** than processes
  - Changing threads is much **faster** than changing processes
    - Hardware support (e.g. Intel's hyperthreading)
  - Creating and terminating threads is also must **faster**



Advantages of Processes over Threads

- Processes offer **isolation**
  - Programming with **threads** is more challenging and complex
    - Need for thread synchronisation (more next week)
  - The OS provides **no protection** on shared resources
    - A thread can read, write, or even erase the stack of another
  - A **bug** in one thread can take the whole process down

- Example Application
  - Chrome uses **one process per tab**
  - Why?

Thread Scheduling

- Threads can be implemented either at **user-level** or at **kernel-level**

- Scheduling user-level threads
  - This **process scheduler** gives a **quantum** to the process
  - The process has a thread that acts as **thread scheduler**
  - The thread scheduler **decides** which thread to run in the quantum
  - Thread scheduling within the quantum is **non-preemptive**

- Scheduling kernel-level threads
  - The **kernel scheduler** schedules **threads** not processes
  - The scheduler knows that **switching threads** within a process costs less than switching threads of different processes

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int *A;

int main(void)
{
  A = malloc(sizeof(int));

  *A = 50;

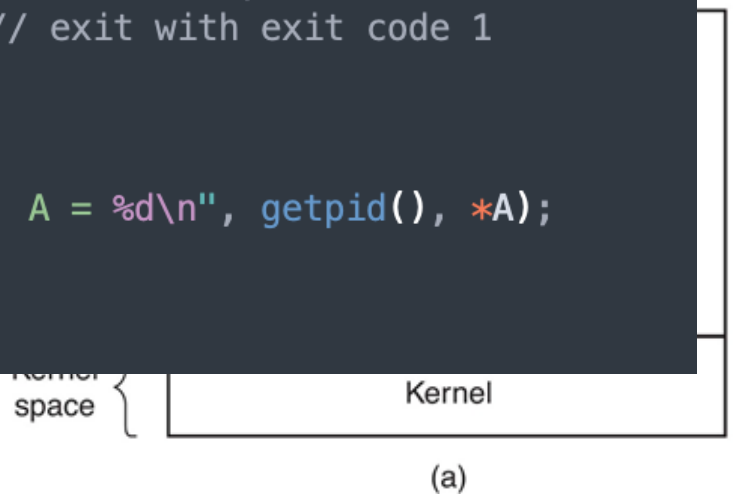
  printf("[%d] I fork.\n", getpid());

  pid_t pid = fork(); // fork a child process
  if (pid == -1) { // error checking
    printf("fork fail\n");
    exit(1); // exit with exit code 1
  }

  *A = *A + 1;
  printf("[%d] A = %d\n", getpid(), *A);

  return 0;
}
```

r of (pse
has a **sir**
rocess h
share t
ties in **di**
s in the



(a)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

int *A;

void* thread(void *input) {
  *A = *A + 1;
  printf("[%d] A = %d\n", getpid(), *A);
  pthread_exit(NULL);
}

int main(void)
{
  pthread_t tid;

  A = malloc(sizeof(int));

  *A = 50;

  printf("[%d] I create a thread.\n", getpid());
  pthread_create(&tid, NULL, thread, NULL);

  *A = *A + 1;
  printf("[%d] A = %d\n", getpid(), *A);

  pthread_join(tid, NULL); // Wait for thread to exit

  return 0;
}
```

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int *A;

int main(void)
{
  A = malloc(sizeof(int));

  *A = 50;

  printf("[%d] I fork.\n", getpid());

  pid_t pid = fork(); // fork a child process
  if (pid == -1) { // error checking
    printf("fork fail\n");
    exit(1); // exit with exit code 1
  }

  *A = *A + 1;
  printf("[%d] A = %d\n", getpid(), *A);

  return 0;
}
```

```
[36464] I fork.
[36464] A = 51
[36465] A = 51
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

int *A;

void* thread(void *input) {
  *A = *A + 1;
  printf("[%d] A = %d\n", getpid(), *A);
  pthread_exit(NULL);
}

int main(void)
{
  pthread_t tid;

  A = malloc(sizeof(int));

  *A = 50;

  printf("[%d] I create a thread.\n", getpid());
  pthread_create(&tid, NULL, thread, NULL);

  *A = *A + 1;
  printf("[%d] A = %d\n", getpid(), *A);

  pthread_join(tid, NULL); // Wait for thread to exit

  return 0;
}
```

```
[36515] I create a thread.
[36515] A = 51
[36515] A = 52
```

Threads

- Multithreading is a **second layer** of (pseudo-)parallelism
  - Traditionally, each process has a **single** thread of control
  - With multithreading, each process has **multiple** threads of control
  - **Threads share the same address space**

```
long counter = 0;

void *worker(void *arg) {
    for (long i = 0; i < 10000000; i++)
        counter++;

    return NULL;
}
```

```
expected: 2000000
actual:   993623
```

